

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	Assessment:	Analytical Problem Solving
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:	Annajothi S	Reg. No.:	192511476
Section / Batch:	2025	Date:	3/8/26

Instructions to Students

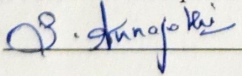
- Identify the appropriate arithmetic algorithm or number representation required for the given computational problem.
- Analyse the step-by-step execution of integer and floating-point arithmetic operations using the selected algorithm.
- Apply Booth's multiplication, restoring/non-restoring division, and IEEE 754 standards to obtain accurate results.
- Compute binary multiplication, division, normalization, and floating-point arithmetic using appropriate algorithms.
- Interpret the computed results by evaluating their accuracy, precision, and suitability for the given application.

QUESTIONS

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyse in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyse how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct

I Annajothi S (192511476) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Marks Evaluation:

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Course Coordinator

Faculty Incharge

Analytical Problem Solving

192511476

A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyse in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits and intermediate steps. Determine whether overflow occurs and explain how the processor detects overflow and interprets the final result.

Binary Conversion

a) Convert +45 to 8-bit binary

$$45_{10} = 00101101_2$$

$$\begin{array}{r} 2 \overline{) 45} \\ 2 \overline{) 22} - 1 \\ 2 \overline{) 11} - 0 \\ 2 \overline{) 5} - 1 \\ 2 \overline{) 2} - 0 \\ 1 - 0 \end{array}$$

b) Convert -23 to 8-bit binary

$$23 = 0001011$$

$$\begin{array}{r} 2 \overline{) 23} \\ 2 \overline{) 11} - 1 \\ 2 \overline{) 5} - 1 \\ 2 \overline{) 2} - 0 \\ 1 - 0 \end{array}$$

-23 $\Rightarrow$ 1's complement of 23

$$11101000$$

$$+ 1$$

$$\begin{array}{r} 11101001 \\ \hline \end{array}$$

$$-23 \Rightarrow 11101001$$

$$45 + (-23)$$

$$\begin{array}{r} 45 \quad 00101101 \\ -23 \quad 11101001 \\ \hline 22 \quad 00010110 \end{array}$$

$$\Rightarrow 22 \Rightarrow 00010110$$

$$45 - (-23)$$

$$\begin{array}{r} 00101101 \\ 00010111 \\ \hline \end{array}$$

$$68 \quad 01000100$$

Operation	Binary Result	Decimal Result	Overflow
$+45 + (-23)$	00010110	22	No
$+45 - (-23)$	01000100	68	No

2) A high performance computing system replaces a ripple carry adder with a 4-bit carry look ahead adder to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processor.

$$A = 1011$$

$$B = 0110$$

$$\text{Initial Carry, } C_0 = 0$$

$$G_i = A_i \cdot B_i, \quad P_i = A_i \oplus B_i$$

	A	B	G	P
1	1	0	0	1
2	1	1	1	0
3	0	1	0	1
4	1	0	0	1

Carry Computation

$$C_1 = G_0 + P_0 C_0 = 0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0 = 1$$

$$C_3 = G_2 + P_2 G_1 + P_2 G_1 G_0 + P_2 P_1 P_0 C_0 = 1$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0 = 1$$

Sum Calculation

$$S_i = P_i \oplus C_i$$

$$1011 + 0110 = 10001_2$$

Comparison with Ripple carry Adder

Ripple Carry Adder	Carry Look - Ahead Adder
Carries propagate one by one.	Carries are computed simultaneously.
Higher delay	Lower delay
Slower for large bit sizes	Faster and suitable for high-speed processor

3) In a signed multiplication operation, a processor uses an algorithm to multiply two numbers, -6 and $+5$. Analyze the algorithm step by step, including the role of the accumulator, multiplier.

Multiplicand (M) = -6

Multiplier (Q) = $+5$

$M = -6 = 1010$

$-M = +6 = 0110$

$Q = +5 = 0101$

Accumulator (A) = 0000

Multiplier (Q) = 0101

Booth Bit (Q_{-1}) = 0

No. of bits = 4

Step	$Q_0 Q_{-1}$	Operation	After ARS
Initial	—	$A = 0000$, $Q = 0101$, $Q_{-1} = 0$	—
1	10	$A = A - M$	ARS
2	11	No Operation	ARS
3	01	$A = A + M$	ARS
4	10	$A = A - M$	ARS

11100010_2

8-bit 2's complement representation of -30

$$(-6) \times (+5) = -30$$

4) A processor is required to perform division of two binary

numbers (eg: $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step by step, showing intermediate remainders and quotient formation.

$$\text{Dividend} = 13 = 1101_2$$

$$\text{Quotient} = 0100_2$$

$$\text{Divisor} = 3 = 0011_2$$

$$\text{Remainder} = 0001_2$$

Restoring Division Algorithm

Initialize Accumulator (A) = 0000 and

Quotient (Q) = 1101

Shift left (A, Q)

n = 4

n	A	Q	Action
4	0000	1101	Initialization
	0001	1010	Left shift (A, Q)
	1110	1010	A = A - M (Negative)
	0001	1010	Restore
3	0011	0100	Left shift (A, Q)
	0000	0100	A = A - M (Positive), Q ₀ = 1
2	0000	1001	Left shift
	1101	1001	A = A - M
	0000	0000	Restore
1	0001	0000	Left shift
	1110	0000	A = A - M
	0001	0000	Restore

Non-Restoring Division.

$$Q = 1101$$

$$M = 0011$$

n	A	Q	Action
4	0000	1101	Initialization
	0001	1010	Left shift (A, Q)
	1110	1010	$A = A - M$
3	1101	0100	Left shift
	0000	0100	$A = A + M$
2	0000	1001	$A = A - M$
1	1101	0010	Left shift
	1011	0010	$A = A - M$
	1110	0010	Left shift
0	0001	0100	$A = A + M$

- 5) A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (eg., 13.25) analyse how it is represented in both single precision and double precision formats, including sign, exponent and mantissa fields.

$$13_{10} = 1101_2$$

$$0.25_{10} = 0.01_2$$

$$13.25 = 1101.01_2$$

$$1101.01_2 = 1.10101 \times 2^3$$

Single Precision

$$\text{Sign bit} = 1 \text{ (-ve)}$$

$$\text{Exponent (8 bits)} = 3 + 127 = 130$$

$$130_{10} = 10000010_2$$

$$\text{Mantissa (23 bit)} = 10101000000000000000000$$

Double Precision

$$\text{Sign bit} = 1$$

$$\text{Exponent (11 bits)} = 3 + 1023 = 1026$$

$$1026_{10} = 1000000010_2$$

Floating Point Addition

$$13.25 + 2.5$$